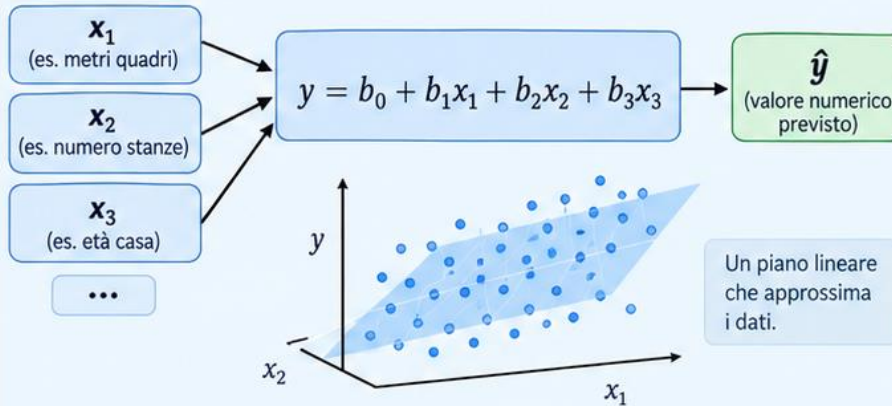


Quattro modi diversi per prevedere un valore numerico

Regressione lineare multivariata

Combina linearmente più variabili di input tramite pesi appresi per prevedere un valore numerico.

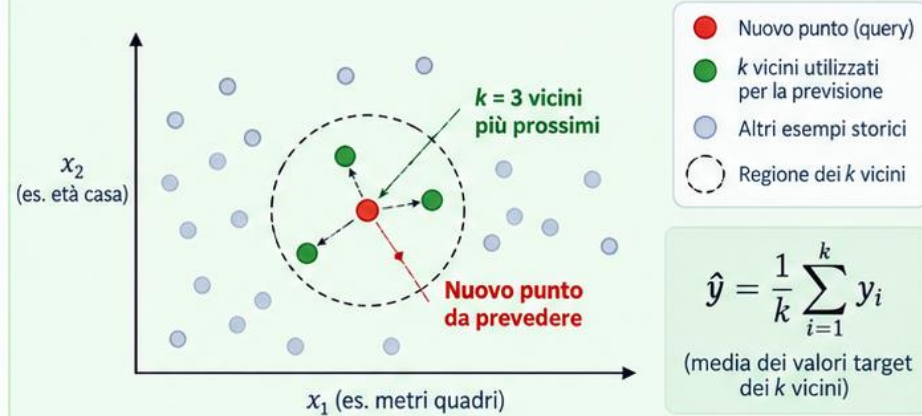


La previsione è una somma pesata delle variabili di input.

link alla demo

KNN

Prevede un valore numerico basandosi sui k esempi storici più simili (più vicini) nello spazio delle feature.

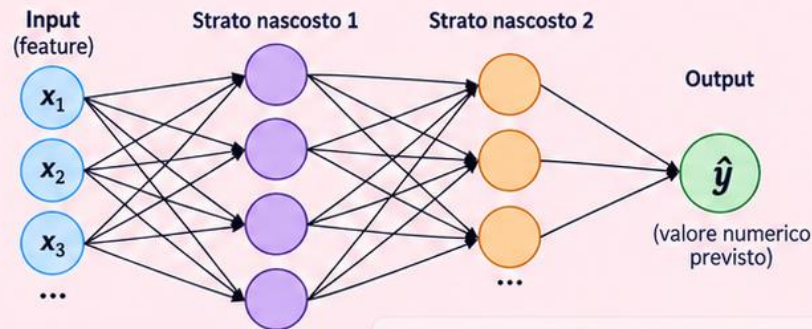


La previsione si basa sui valori osservati di casi simili nello spazio delle feature.

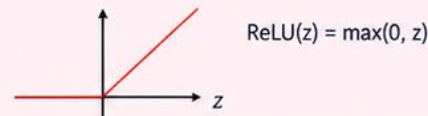
link alla demo

Rete neurale MLP

Una rete di neuroni (strati di nodi) che apprende relazioni non lineari tra input e output.



Le funzioni di attivazione (es. ReLU) permettono di modellare relazioni non lineari.

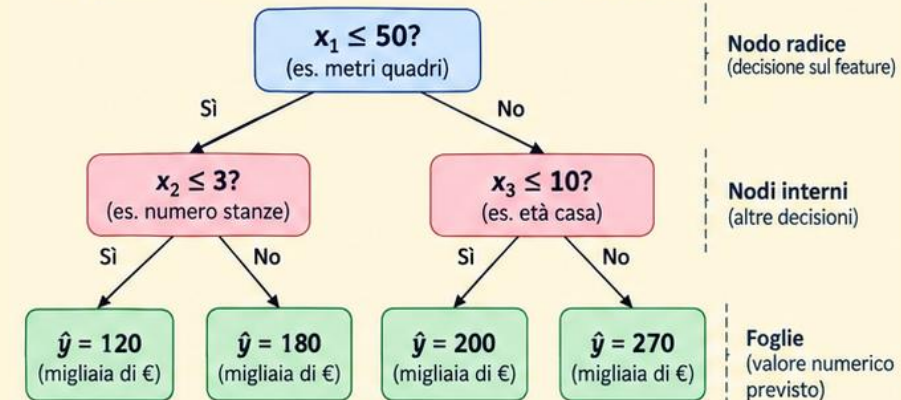


La rete apprende automaticamente rappresentazioni complesse dei dati.

link al playground di TensorFlow

Albero decisionale

Divide lo spazio delle feature in regioni tramite semplici regole e assegna un valore numerico in ciascuna regione (foglia).



La previsione è il valore associato alla foglia raggiunta seguendo le regole.

link alla demo

link alla
demo

Le componenti della regressione

predictor, 'x-variable',
independent variable,
explanatory variable

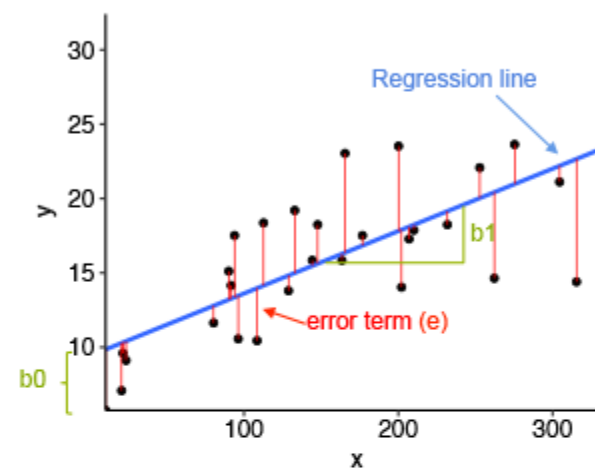
coefficient

$$Y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \varepsilon$$

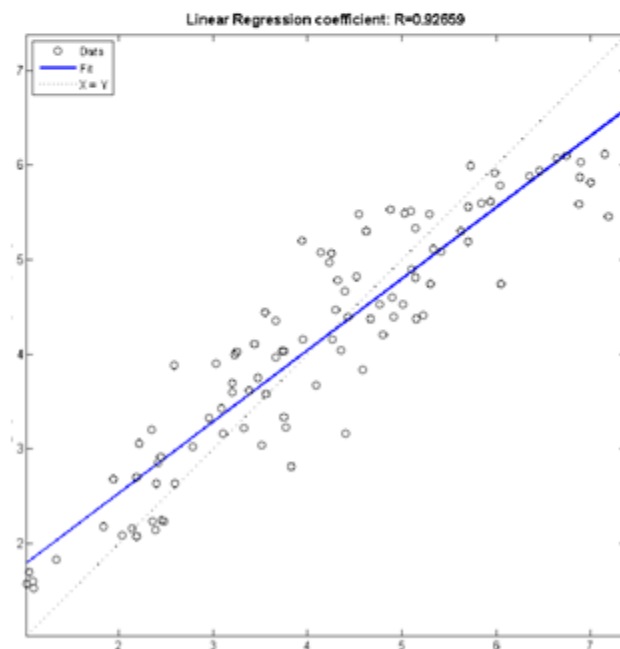
linear predictor

response, dependent variable,
observation, 'y-variable'

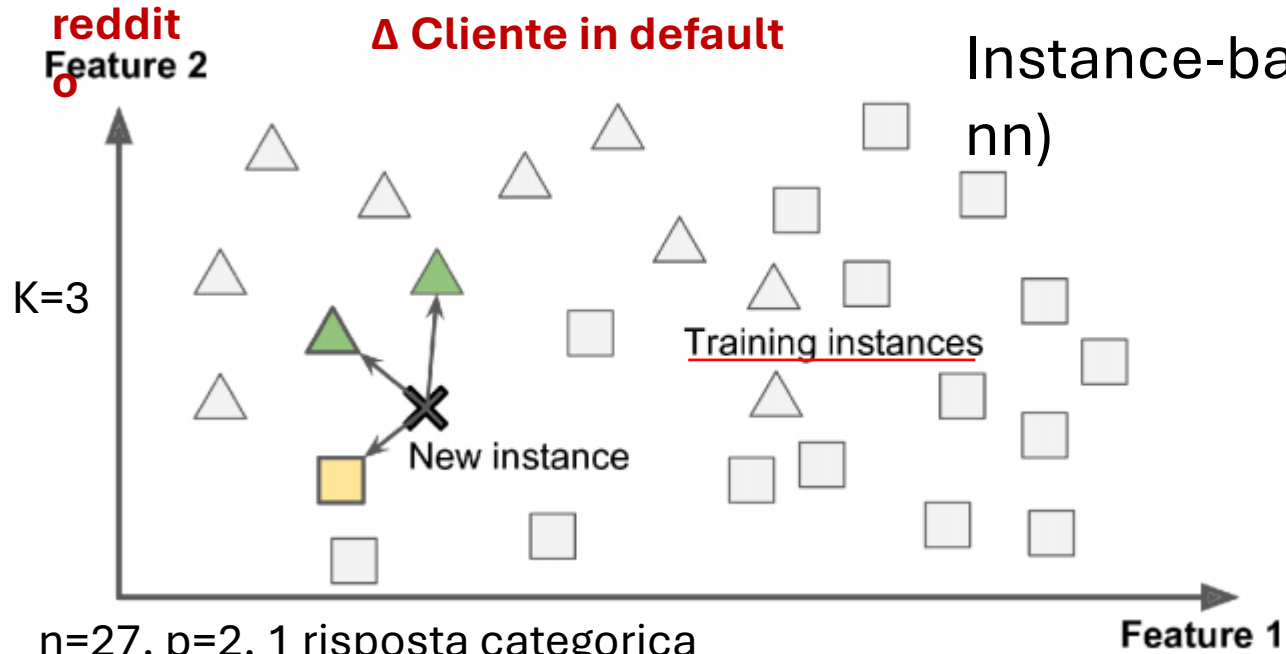
random error,
"noise"



nel caso univariato

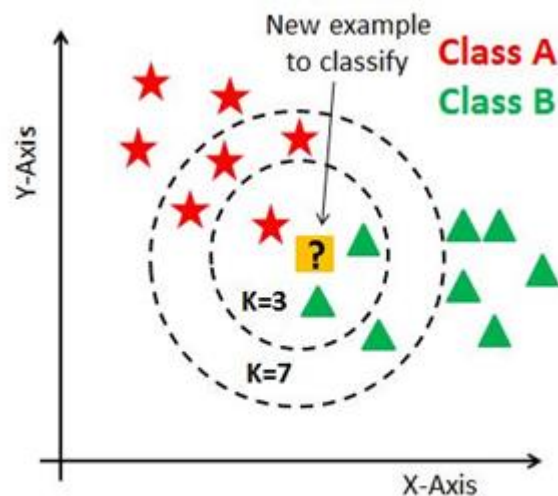


Retta di regressione
vera vs stimata



$n=27$, $p=2$, 1 risposta categorica (quadrato/triangolo)

K-nn è un algoritmo *lazy* (pigro) perché rimanda il calcolo al momento in cui è nota la nuova istanza da prevedere (in altre parole, non c'è un vero training)



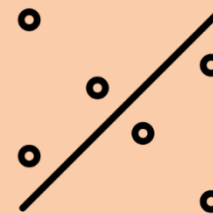
K=3 → triangolo
K=7 → stella
K=15 → triangolo

$n=15$, $p=2$, 1 risposta categorica stella/triangolo)

Non nel K-nn!

01 Model-based Learning

al contrario: Model-based learning algorithms use the training data to create a model that has parameters learned from the training data. For example: In Support Vector Machines aka SVM, we have w^* (learned weights value) and b^* (learned bias value). After the model is built, the training data can be discarded.



Una rete *feed-forward fully connected* a livelli multipli (per MNIST)

MLP ($K_1 = 256, K_2 = 128$)

In questa rete qui raffigurata dobbiamo POI inserire (in genere alla fine) la funzione di attivazione e la funzione *softmax*

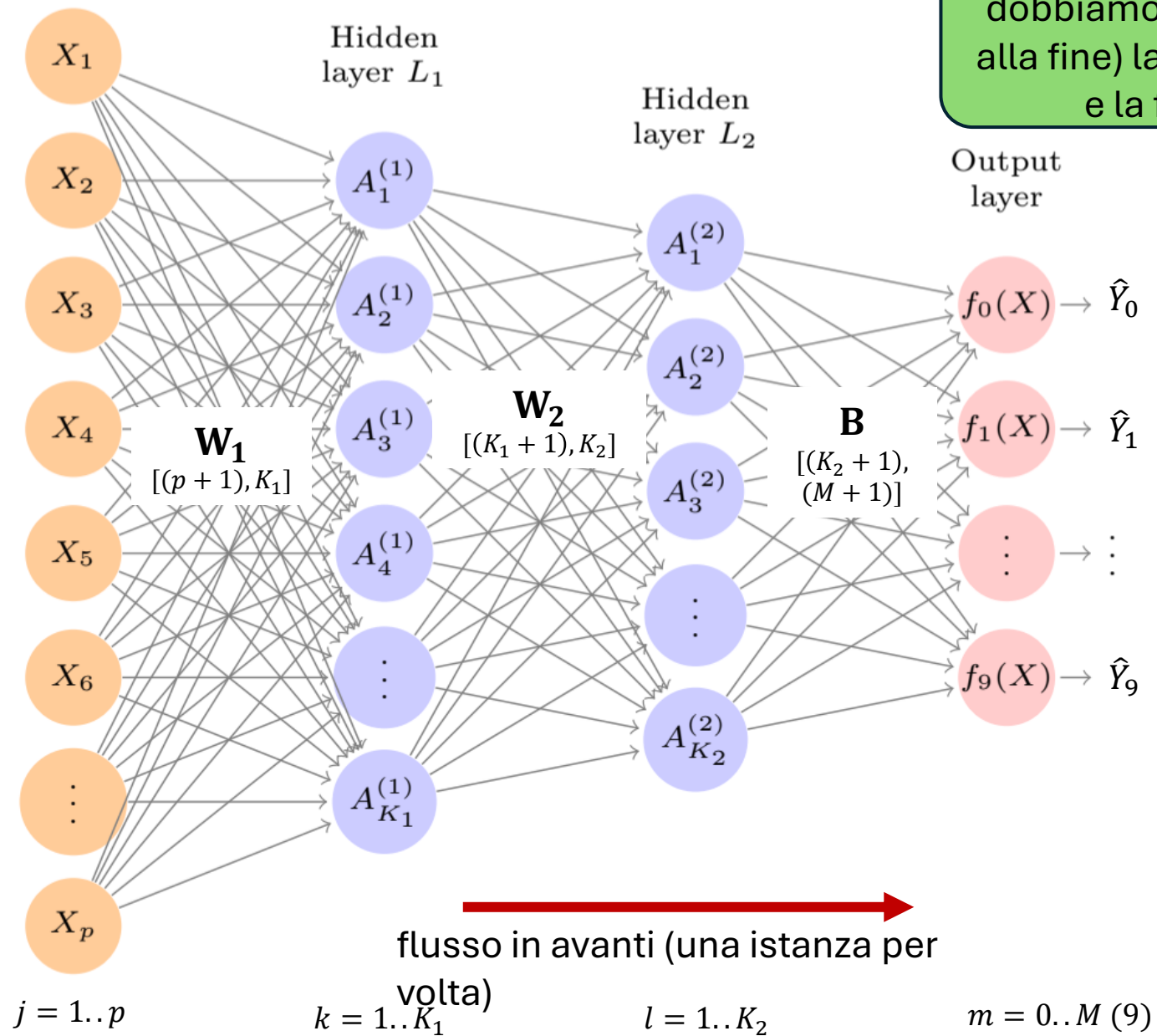
La matrice $\mathbf{W}_1$ contiene tutti i coefficienti che alimentano il primo livello nascosto L_1 dal livello di input.

La matrice $\mathbf{W}_2$ contiene tutti i coefficienti che alimentano il secondo livello nascosto L_2 dal primo livello nascosto L_1 .

La matrice $\mathbf{B}$ contiene tutti i coefficienti che alimentano il livello di output dal secondo livello nascosto L_2 .

La prima dimensione di queste tre matrici è aumentata di 1 per tenere conto dei **bias** (qui non mostrati per semplicità)

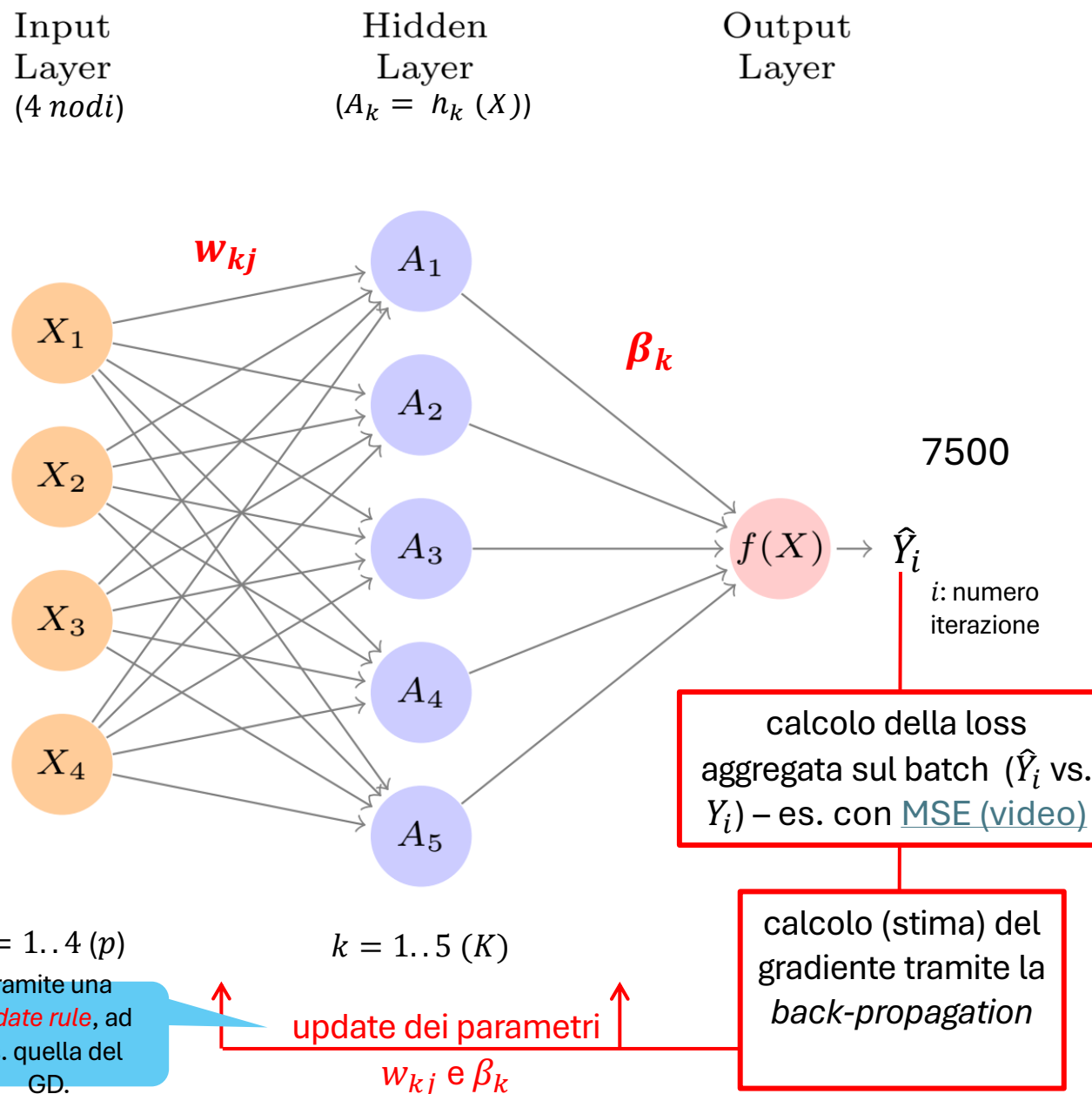
Non interpretare le notazioni $\mathbf{W}_i$ e $\mathbf{B}$ per weight e bias. Tutte le matrici contengono sia weight che bias.



le 10 variabili numeriche $\hat{Y}_m$ rappresentano una singola variabile qualitativa

La matrice dei pesi $\mathbf{W}$ di ogni layer è anche detta la «kernel matrix»

Il processo iterativo (mediante *feedback-closed loop*)



Ad ogni iterazione (batch dell'epoca) i -esima, ❶ il calcolo della loss (aggregata), ❷ la stima del gradiente in quel punto (cioè, con quel set di parametri) e ❸ **l'update dei parametri** (seconda la regola dell'algoritmo) si fanno:

- per ogni riga del dataset di training, dopo che essa è passata attraverso la rete (*case updating, aka on-line updating*); in questo caso $\hat{Y}_i$ è uno scalare;
- alla fine dell'«epoca», cioè dopo che tutte le righe sono passate attraverso la rete (*batch updating, non necessariamente full*); in questo caso $\hat{Y}$ è un vettore;
- in genere il numero di update è dato da:

$$(n/\text{batch-size}) \times \text{\#epoche}$$

(Jason Brownlee – MLM)

Il case updating tende a fornire previsioni più accurate, ma è troppo lento in esecuzione per grandi dataset di training o semplicemente quando le epoche sono centinaia o migliaia; è di fatto abbandonato (a meno di stream input);

Ci sono diversi modi per calcolare l'errore (diverse *loss function*) e diversi modi per modificare i parametri.

NB. Una rete è *feed-forward* (senza cicli), come questa, quando al suo interno può essere percorsa solo in avanti, a differenza delle reti *ricorsive*, che ammettono loop.

chat su come interpretare
il problem type
«regression» (al fondo)

Progettazione GRAFICA di una rete neurale – 1/5: playground di TensorFlow

Epoch
000,000

Learning rate
0.03

Activation
ReLU

Regularization
None

Regularization rate
0

Problem type
Classification

DATA

Which dataset do you want to use?

4 dataset con segnale più o meno complesso da riconoscere e separare in classi

Ratio of training to test data: 60%

Noise: 15

%rumore – sopra 50% non ha senso prevedere

Batch size: 10

REGENERATE

FEATURES

Which properties do you want to feed in?

X_1

X_2

X_1^2

X_2^2

X_1X_2

$\sin(X_1)$

$\sin(X_2)$

This is the output from one neuron. Hover to see it larger.

6 HIDDEN LAYERS

5 neurons

3 neurons

3 neurons

2 neurons

2 neurons

2 neurons

The outputs are mixed with varying weights, shown by the thickness of the lines.

OUTPUT

Test loss 0.513
Training loss 0.511

Le due loss non sono qui necessariamente le % di misclassificazione

Colors shows data, neuron and weight values.

dati di test con bordo

Da qualche prova empirica con il dataset posizione 2 ed il rumore a 20, sembra che la loss di test più bassa si ottenga con 1 layer nascosto di 8 nodi, la tanh e la regolarizzazione L1. Selezionando tutte le trasformazioni dei predittori riusciamo a fittare bene anche il dataset più complesso

Nella realtà segnale e rumore non sono sotto il nostro controllo, perché fanno processo del fenomeno

link alla
demo

Alberi di classificazione

n=32, p=3

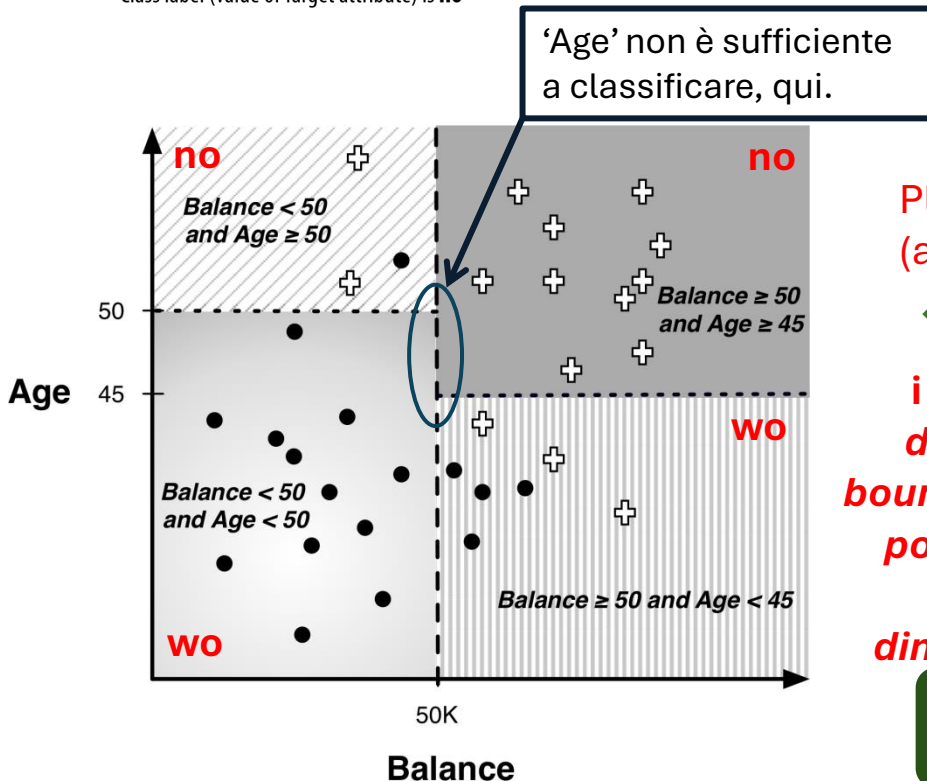
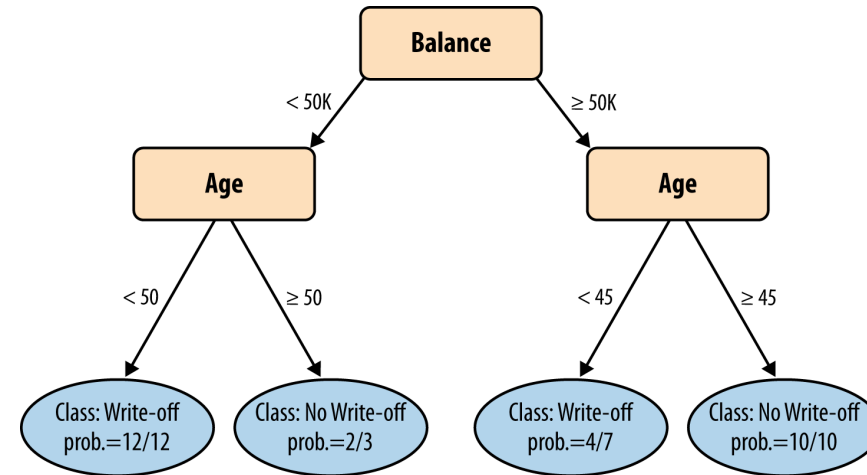
Attributes				Target attribute
Name	Balance	Age	Employed	Write-off
Mike	\$200,000	42	no	yes
Mary	\$35,000	33	yes	no
Claudio	\$115,000	40	no	no
Robert	\$29,000	23	yes	yes
Dora	\$72,000	31	no	no

This is one row (example).

Feature vector is: **<Claudio,115000,40,no>**

Class label (value of Target attribute) is **no**

fitting



Plotting
(a valle)



**i relativi
decision
boundary (i cut
point delle
varie
dimensioni)**

Ogni albero ha una
rappresentazione
grafica ed una a regole

If Balance < 50K
If Age < 50 then assign 'wo'
else assign 'no';
Else
If Age < 45 then assign 'wo'
else assign 'no'.

Animazione di alberi

Fonte: DSfB (Provost & Fawcett)

Alberi di classificazione

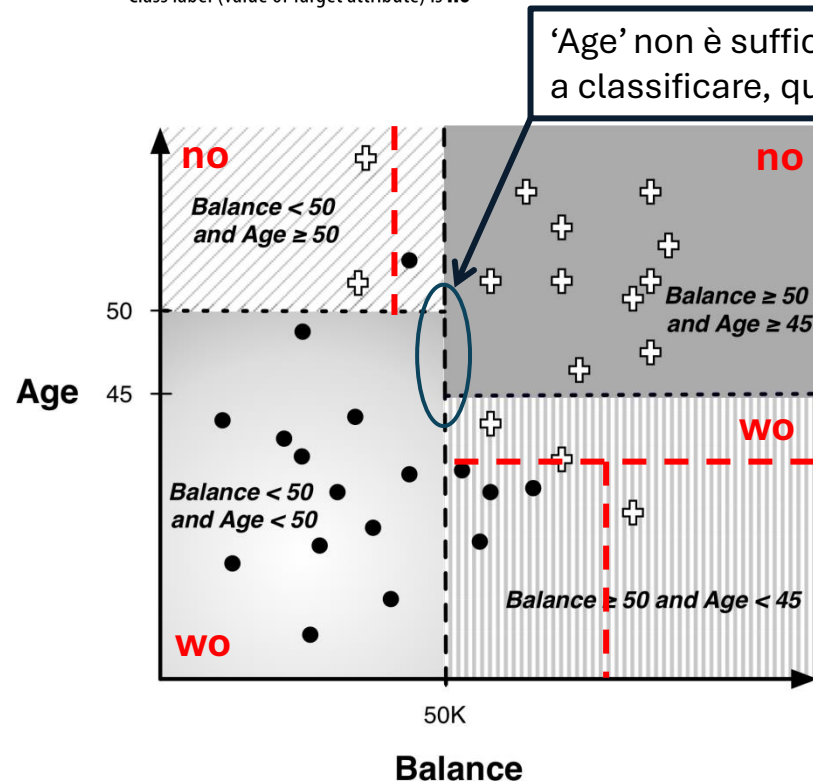
n=32, p=3

Attributes				Target attribute
Name	Balance	Age	Employed	Write-off
Mike	\$200,000	42	no	yes
Mary	\$35,000	33	yes	no
Claudio	\$115,000	40	no	no
Robert	\$29,000	23	yes	yes
Dora	\$72,000	31	no	no

This is one row (example).

Feature vector is: **<Claudio,115000,40,no>**

Class label (value of Target attribute) is **no**



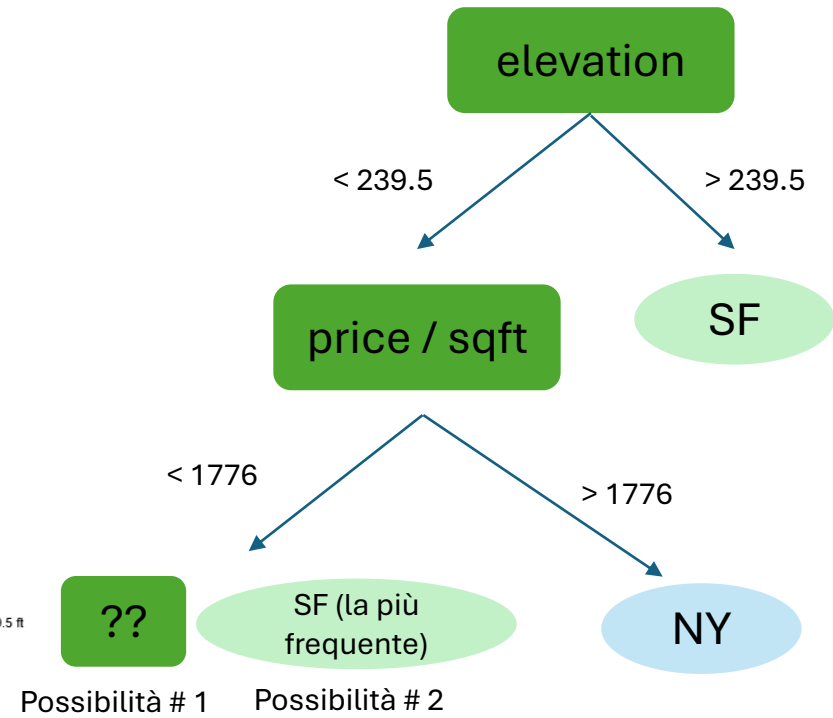
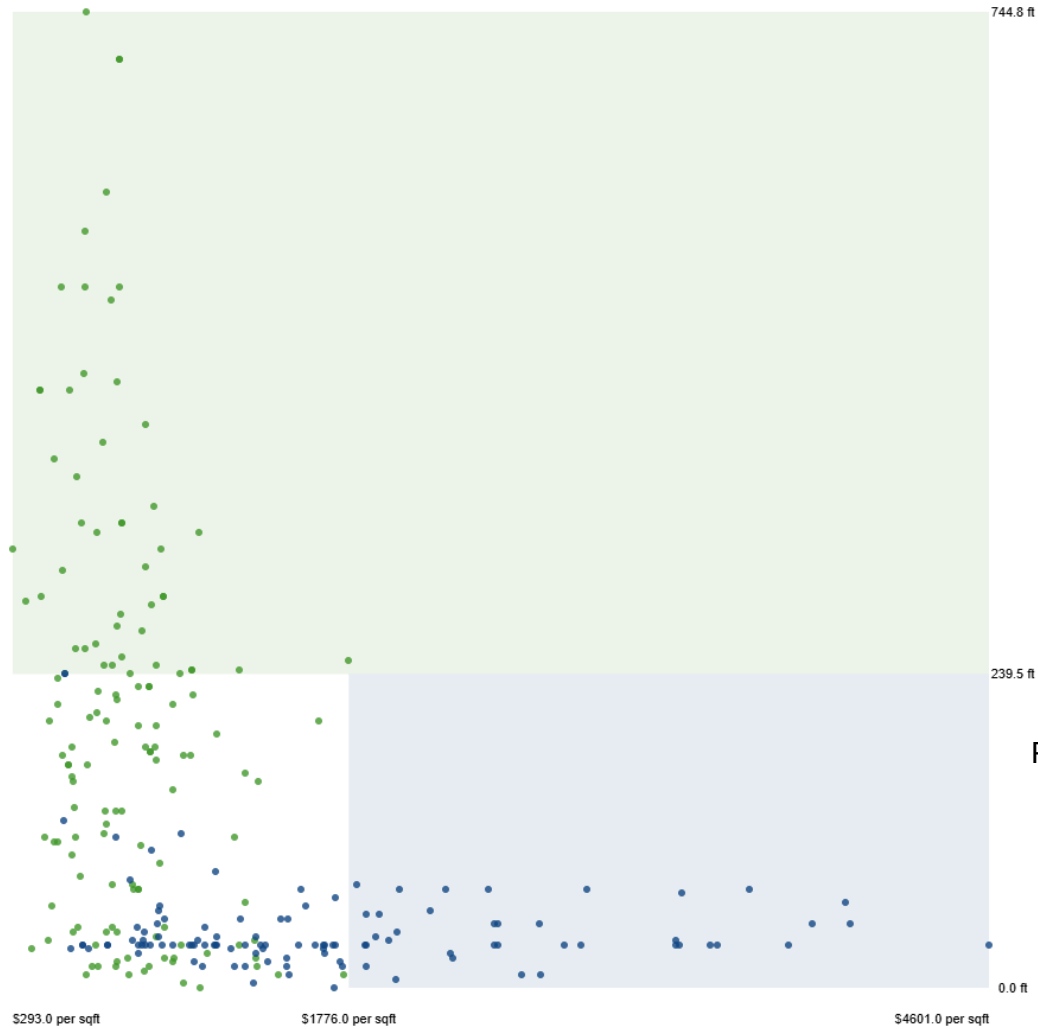
fitting

Plotting
(a valle)

**i relativi
decision
boundary (i cut
point delle
varie
dimensioni)**

```
If Balance < 50K
  If Age < 50 then assign 'wo'
  else
    if Balance < 40 then assign 'no'
    else assign 'wo';
Else
  If Age < 45 then
    if Age < 35 then assign 'wo'
    else
      if Balance < 80 then assign 'wo'
      else assign 'no'
  else assign 'no'.
```

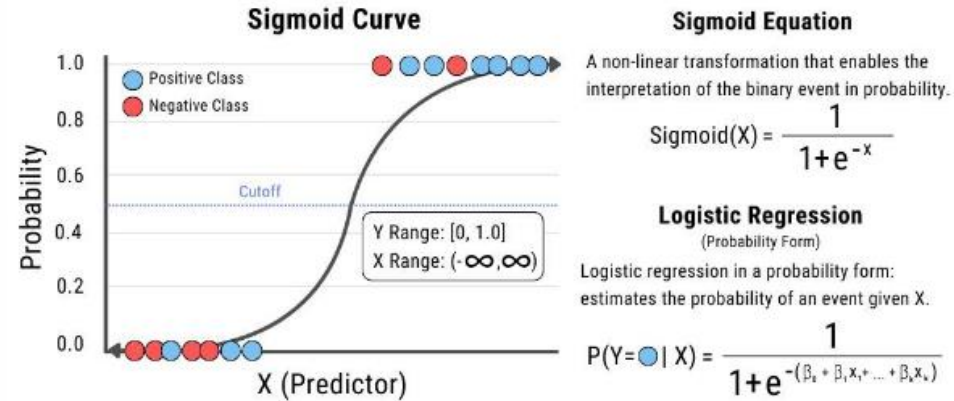
Fonte: DSfB (Provost & Fawcett)



Logistic Regression

Core Concept in Statistics

DataInterview.com



Logistic Regression

A statistical model that uses a **logistic function** to model the probability of a binary outcome based on a set of predictor variables.

Logistic Regression (Linear Form)

$$y = \log\left(\frac{P(Y = \text{Positive Class} | X)}{P(Y = \text{Negative Class} | X)}\right) = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k$$

Outcome Variables Log Link Function Odds Constant Predictor Variables

Logit Regression Results

Dep. Variable:	y	No. Observations:	100
Model:	Logit	Df Residuals:	97
Method:	MLE	Df Model:	2
Date:	Mon, 08 Jul 2024	Pseudo R-squ.:	0.01629
Time:	18:05:18	Log-Likelihood:	-68.185
converged:	True	LL-Null:	-69.315
Covariance Type:	nonrobust	LLR p-value:	0.3232

	coef	std err	z	P> z	[0.025	0.975]
const	-0.0428	0.204	-0.210	0.834	-0.443	0.358
x1	-0.0201	0.198	-0.101	0.919	-0.409	0.368
x2	0.2987	0.202	1.477	0.140	-0.098	0.695